

fastPLS: a memory-aware accelerated SIMPLS implementation for high-dimensional biomedical data with optional CUDA and Metal routes

Dupe Ojo^{a,†}, Alessia Vignoli^{b,c,†}, Stefano Cacciatore^{a,d,*}, Leonardo Tenori^{b,c,*}

^a Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology, Cape Town 7925, Western Cape, South Africa

^b Department of Chemistry “Ugo Schiff”, University of Florence, Sesto Fiorentino, Italy

^c Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy

^d Department of Integrative Biomedical Sciences, Institute of Infectious Disease and Molecular Medicine, University of Cape Town, Cape Town 7925, South Africa

†These authors contributed equally and share first authorship.

*Co-corresponding authors: Stefano Cacciatore, stefano.cacciatore@icgeb.org; Leonardo Tenori, tenori@cerm.unifi.it.

Abstract

Background and objective: High-dimensional biomedical PLS workflows, including multivariate NMR prediction and repeated validation, can be limited by the sequential cost and storage of SIMPLS. We developed fastPLS to accelerate SIMPLS while retaining de Jong's component equations.

Methods: The reviewed public interface is fastPLS 0.99.25; analysis-specific executing archives and commits are mapped in the Supplement. The implementation uses compiled, shape-dependent execution, incremental coefficient and fitted-value updates, compact latent prediction,

and optional implicit cross-covariance products. Fixed-control CPU IRLBA, an iterative truncated solver rather than an exact dense decomposition, was the deterministic numerical reference. Approximate rSVD and accelerator routes were audited separately under prespecified criteria.

Results: fastPLS SIMPLS met the prespecified deterministic numerical tolerances in all 117 component-level comparisons. In 108 repeated single-CPU argmax comparisons with `pls::simpls.fit`, accuracy was identical throughout; fastPLS was faster on five of nine datasets under ordinary public workflows, with a largest speed-up of 4.84-fold. LDA results were evaluated separately as complete-workflow comparisons. For the 13,000 by 28,355 NMR predictor-response problem at 50 SIMPLS components, qualified CPU and CUDA rSVD reduced the fixed-control IRLBA runtime from 350.7 s to 9.80 and 1.51 s, respectively, while meeting the prespecified numerical tolerances.

Conclusions: fastPLS reduces the computational barrier to sequential SIMPLS in large biomedical workflows while making solver and backend qualification explicit.

Keywords: partial least squares; accelerated SIMPLS; randomized SVD; CUDA; Metal; metabolomics

1. Introduction

Partial least squares (PLS) combines supervised dimension reduction with prediction and is widely used when biomedical predictors are correlated, high dimensional, or accompanied by multivariate responses [1-3]. Applications include cancer metabolomics and prediction of multiple NMR spectra from one acquisition [4-7].

Increasing spectral resolution, single-cell sample counts, foundation-model embeddings, and repeated validation make PLS computationally demanding. KODAMA, for example, repeatedly fits cross-validated PLS discriminant models [8,9]. UNI and Prov-GigaPath illustrate the related post-extraction matrix regime in computational pathology [31,32]. The ImageNet/DINOv2 stress test assessed reproducible scalability, not biomedical validity [28,29].

Among PLS formulations, SIMPLS constructs sequential components without explicitly deflating the predictor matrix [11]. PLS-SVD provides a one-shot cross-covariance comparator [10], while OPLS and kernel PLS extend the framework to orthogonal filtering and nonlinear relations [12-14]. Reference software such as `pls::simpls.fit` already returns coefficient and fitted-value arrays for every component prefix; fastPLS does not claim otherwise. Its distinct execution features are compact latent prediction, shape-dependent intermediate storage, optional implicit cross-covariance products, compiled low-rank solvers, and route-qualified CUDA and Metal support. Supplementary Table S1a defines this feature comparison explicitly.

We present fastPLS, whose principal methodological contribution is a compiled, shape-dependent execution of de Jong SIMPLS. Sequential deflation, orthogonalization, and component definitions are retained, while avoidable intermediate products and dense outputs are reduced according to matrix shape and requested output. Low-rank solvers, implicit products, float32, cross-validation, LDA, and CPU/CUDA/Metal execution are supporting options around this core estimator. OPLS and kernel PLS reuse the same optimized SIMPLS engine. The GPL-3 R package uses reusable components from the MIT-licensed kodama-cpp project.

2. Materials and methods

2.1 Software architecture

The public `pls()` interface selects PLS family, component count, solver, backend, and, for classification, `argmax` or latent-space linear discriminant analysis (LDA). `pls.single.cv()` selects settings within one cross-validation layer; `pls.double.cv()` uses nested cross-validation. The audited 0.99.25 namespace explicitly exports `pls()`, `pls.single.cv()`, `pls.double.cv()`, `evaluate()`, `plot.permutation()`, `ViP()`, `fastsvd()`, `fastcor()`, `fastPLS_backend()`, `has_cuda()`, and `has_metal()`; registered `predict()` and `plot()` methods use the standard R generics. The supported model families are PLS-SVD, SIMPLS, OPLS, and kernel PLS; the public classifiers are `argmax` and LDA; and `rSVD` is the default solver, whereas IRLBA is CPU-only. PCA and nearest-neighbour classifiers are not package APIs. The legacy `lda_ridge` argument is deprecated and ignored, emits a warning when supplied, and is excluded from cross-validation tuning because LDA uses a fixed scale-normalized Cholesky fallback sequence. Requested estimators and unavailable backends are never silently substituted. Route status and host/device residency are defined in Supplementary Tables S1 and S9.

Let a index a component. A denotes the maximum retained component count, C the requested component-count set, and K denotes the number of cross-validation folds. Lowercase k is reserved for retrieval cutoffs.

The architecture separates preprocessing, the PLS estimator, direction extraction by IRLBA or `rSVD`, and CPU/CUDA/Metal execution (Figure 1). Benchmark rows recorded requested and executed estimators and rejected mismatches.

Figure 1. fastPLS architecture separating response representation, PLS estimator, low-rank solver, backend, and prediction head. Accelerator routes may be native or explicitly reported as hybrid.

CPU routines use compiled C/C++ and R-linked BLAS/LAPACK; CUDA uses NVIDIA CUDA/cuBLAS and Metal uses Apple Metal Performance Shaders. The external comparison used one effective BLAS thread, so no multicore speed-up is claimed. Versions, build flags, residency, and thread settings are in Supplementary Tables S1 and S4a.

2.2 Shape-aware SIMPLS execution and related PLS models

Predictors were centred and optionally scaled using training statistics; numeric responses were centred. Classification used a multivariate PLS-DA response, with class-wise products replacing dense one-hot matrices on large-class routes.

The accelerated method targets SIMPLS; PLS-SVD is retained as a one-shot comparator. Both operate on the centred cross-covariance

$$S = X^T Y.$$

PLS-SVD decomposes the centred cross-covariance once at the largest valid requested rank and reuses prefixes. Its component count is bounded by cross-covariance rank and, for C centred classes, by C-1.

SIMPLS follows de Jong's sequential score, loading, orthogonalization, and rank-one deflation equations [11]. As in `pls::simpls.fit`, one fit supplies the standard sequential path through all

components up to the requested maximum; fastPLS does not claim this path construction as novel. Its contribution is computational and shape dependent: rank-one deflation products are reused; coefficient and fitted-value quantities are updated incrementally; X-transpose-X is cached only when its setup and storage can be amortized; compact prediction retains latent factors instead of dense coefficient and prediction paths; and implicit products avoid an explicit predictor-response cross-covariance when that matrix is limiting. Every component invokes a fresh rank-one direction calculation on the current deflated state. Candidate-direction blocks, cross-component warm starts, and adaptive refresh were rejected and are not used by the public CPU, CUDA, or Metal paths.

$$S_{a+1} = S_a - v_a (v_a^T S_a).$$

A shape-aware route can cache X-transpose-X for score normalization and loading calculations. Compact latent factors and blocked prediction avoid retaining dense coefficient or prediction paths. Optional implicit products and rSVD change how directions are computed or stored, not the SIMPLS deflation equations.

Algorithm 1 summarizes the unchanged estimator and the retained execution updates. The minimally optimized compiled baseline, asymptotic costs, storage terms, and detailed de Jong mapping are provided in Supplementary Table S2 and Section S12.1.

Algorithm 1. Shape-aware SIMPLS execution. Direction extraction uses deterministic IRLBA or approximate rSVD; score construction, orthogonalization, and deflation follow de Jong [11].

Input: centred X ; numeric, indicator, or label-aware response Y ; maximum component count A ; requested component-count set C ; solver.

1. Define $S_0 = X^T Y$ explicitly, or define operator products $S(v) = X^T(Yv)$ and $S^T(u) = Y^T(Xu)$; label-aware products use class sums.
2. If $p \leq n$ and storage permits, cache $G = X^T X$. Initialize R , Q , and V as empty; set fitted values $\hat{Y}_0 = 0$.
3. For component $a = 1, \dots, A$:
 - 3.1 Extract the leading right direction g_a of state S_{a-1} using a fresh IRLBA solve or a fresh oversampled rSVD sketch.
 - 3.2 Set $r_a = S_{a-1} g_a$ and $t_a = X r_a$; divide r_a and t_a by $\|t_a\|_2$.
 - 3.3 Compute predictor and response loadings $p_a = X^T t_a$ and $q_a = Y^T t_a$.
 - 3.4 Orthogonalize $v_a = (I - VV^T)p_a$ and normalize v_a .
 - 3.5 Deflate $S_a = S_{a-1} - v_a(v_a^T S_{a-1})$, or update the equivalent implicit operator.
 - 3.6 Append r_a , q_a , and v_a ; update $B_a = B_{a-1} + r_a q_a^T$ and $\hat{Y}_a = \hat{Y}_{a-1} + t_a q_a^T$.
 - 3.7 If $a \in C$, store only the requested coefficient/prediction snapshot or compact latent representation.

Output: the standard sequential component path; requested outputs are retained as dense snapshots or compact latent factors.

OPLS and kernel PLS are secondary extensions that reuse the accelerated SIMPLS core. OPLS first applies an orthogonal predictor filter [12]. Linear kernel PLS dispatches to the linear route;

RBF and polynomial kernels form an n-by-n Gram matrix [13,14], restricting nonlinear models to moderate sample sizes.

2.3 Truncated SVD and memory-aware computation

Direction extraction is modular rather than the principal estimator contribution. float64 CPU fitting supports fixed-control IRLBA [15] as the deterministic numerical reference and approximate rSVD [16], which uses a Gaussian range sketch, orthonormalization, power iterations, and a reduced decomposition. IRLBA is an iterative truncated solver and is not described as an exact dense SVD. In release 0.99.23, the rSVD default was changed to oversampling 20 and two power iterations, the CPU and CUDA setting that met every prespecified check across five seeds. Explicit unqualified overrides warn and are recorded in model diagnostics. Exact controls, seeds, audit thresholds, and route-specific qualification are consolidated in Supplementary Sections S13-S14 and Tables S8-S9.

For selected large shapes, fastPLS avoids materializing S and evaluates the same operator through

$$S\Omega = X^T(Y\Omega), S^TQ = Y^T(XQ).$$

When explicit cross-covariance is large, identical centred/scaled operators are evaluated as X-transpose-(YV) and Y-transpose-(XU). Class-wise sums avoid dense indicator responses.

Blocking changes storage, not the global operator.

Compact latent prediction processes large test matrices in row blocks and releases temporary scores after each block. For the ImageNet top-5 analysis, predictions were similarly blocked and classification counts were accumulated online, avoiding a full 281,167-by-1,000 double score matrix.

2.4 CPU, CUDA, Metal, and precision

Standard R matrices use eight-byte float64 values; float-package inputs use four-byte float32 values. float64 is the reference. Precision support and backend residency are route specific; the authoritative capability classifications are in Supplementary Tables S1 and S9. On Windows, a portable float-package CPU fallback supports rSVD PLS-SVD, SIMPLS, and linear-kernel SIMPLS with argmax, whereas native compiled float32 OPLS, nonlinear kernel PLS, and LDA remain unavailable.

2.5 Prediction, classifiers, and validation

Regression returns continuous predictions. Classification uses argmax PLS-DA or LDA fitted to PLS scores. LDA uses pooled within-class covariance, Cholesky solves, class priors, and deterministic trace-scaled regularization only when factorization fails [17,18]. For imbalanced classification, cross-validation can select by balanced accuracy, defined as the unweighted mean of class-specific recalls; nested permutation testing uses the same selected endpoint rather than substituting dummy-response Q2. Permutation inference uses the finite Monte Carlo correction $(b+1)/(B+1)$, where b is the number of successful null statistics at least as extreme as observed and B is the number of successful null fits. Independent observations are permuted by row; repeated observations identified by the grouping constraint are exchanged as complete blocks within equal-size exchangeability strata. Group sizes, within-group responses, and class frequencies are thereby preserved, while fold assignments and randomized-solver seeds remain fixed across observed and null analyses. Failed null fits are excluded from B and reported explicitly.

$$\Sigma = \{ T^T T - \sum_c n_c \mu_c \mu_c^T \} / \max(1, n - C).$$

$$\delta_c(t) = t^\tau w_c - \frac{1}{2} \mu_c^\tau w_c + \log(n_c/n).$$

Fold construction, fitting, prediction, and metric accumulation remain compiled where supported; grouped observations can be constrained to one fold. Model-selection endpoints include accuracy, balanced accuracy, training R^2 , cross-validated Q^2 , and RMSD. Training R^2 uses fitted training responses and the complete-training response mean. Independent-test Q^2 uses test prediction error but centers the denominator on the training-response mean. Cross-validated Q^2 instead sums fold prediction errors and centers each held-out fold on the mean estimated from that fold's training observations. For PLS-DA, the same R^2 and Q^2 formulas are applied to dummy-coded responses; these quantities are distinct from decoded-label accuracy and balanced accuracy. Hybrid OPLS, nonlinear-kernel, and Metal paths are identified explicitly.

2.6 Benchmark and reproducibility design

Twelve tasks covered metabolomics, NMR, CITE-seq, tissue and cancer omics, single-cell transcriptomics, drug response, and CIFAR-100 [7,20-27,30]. Methods used identical stored splits and training-selected component grids. CIFAR-100 followed its documented 50,000/10,000 split [30]. Classification used accuracy and Wilson intervals; multivariate regression used RMSD, independent-test Q^2 relative to the training-response mean, and held-out bootstrap intervals. Runtime included fitting and prediction. Absolute host RSS was the isolated R process high-water mark. A pre-fit baseline was recorded after package/data preparation and garbage collection; the baseline-corrected process increment was peak minus baseline. This increment still contains returned outputs, temporary BLAS allocations, allocator effects, and benchmark/runtime state, and is not labelled algorithmic workspace. Formula-based dense-object

sizes were reported separately. Process-specific GPU increments similarly include runtime context rather than workspace alone.

A separate exploratory ImageNet/DINOv2 analysis used a fixed 1,000,000/281,167 development split of 1,024-dimensional embeddings. The split was noncanonical and previously informed component choices. SIMPLS argmax/LDA classification and raw/PCA/PLS FAISS retrieval were therefore treated as separate exploratory experiments.

Five-fold training-only selection was performed separately by PLS family; boundary selections were described as best within the grid. Accelerator speed-up was interpreted only for numerically concordant paired predictions. CUDA timing covered data transfer, execution, synchronization, and returned model or predictions; exact concordance thresholds and timing boundaries are reported in the Supplement.

A controlled one-factor-at-a-time SIMPLS study varied training observations (500-10,000), predictors (50-2,000), responses (10-1,000), retained components (2-80), requested prefixes (1-20), exact cross-covariance rank (5-80), classes (5-200), and explicit cross-covariance storage (2-64 MB). Each point used three matched seeds, a deterministic float64 CPU-IRLBA reference, and rSVD with oversampling 20 and two power iterations. Fresh processes recorded fit and prediction time, baseline-corrected peak host RSS, CUDA memory, and numerical agreement; speed crossovers were inferred only when every replicate met tolerance.

The primary evidence tested whether deterministic IRLBA SIMPLS met the prespecified numerical tolerances against de Jong SIMPLS and improved runtime. rSVD was assessed separately using computational screens covering prediction error and correlation, latent-subspace agreement, decoded labels, and endpoint metrics. The definitive default-control audit used seeds

1, 7, 19, 43, and 123. These screens are engineering guardrails rather than clinical-equivalence margins, and qualifying a control configuration does not certify every future dataset-specific fit. Their exact definitions, tolerances, discordant counts, and audit results are reported in Supplementary Sections S12-S14 and Tables S7-S9. Deterministic IRLBA remains the confirmatory route when individual prediction changes are consequential.

External timing used two prespecified profiles. The minimum-output profile compared deterministic float64 SIMPLS while requiring the complete coefficient path, centering quantities, and final held-out predictions; scores, loadings, fitted-value arrays, and variance summaries were suppressed, with `pls::simpls.fit` called using `stripped = TRUE`. The public-workflow profile retained each implementation's ordinary model object and final predictions. Package and data loading preceded timing, no numerical warm-up was used, and every method-dataset repetition ran in a fresh process with one effective BLAS thread and a common 10,000-s timeout. Three repetitions were attempted for every pair; medians, IQRs, completed repetitions, failures, object sizes, and process RSS were retained.

3. Results

Results are organized around the shape-dependent SIMPLS execution contribution. Primary evidence comprises deterministic estimator comparison and the matched IRLBA comparison with independent implementations. rSVD, precision, and hardware results are reported separately with their numerical-audit status; the NMR and ImageNet large-scale analyses are exploratory feasibility studies rather than confirmatory evidence.

Deterministic fastPLS SIMPLS met the prespecified numerical tolerances in all 117 component-level comparisons with de Jong SIMPLS. Approximate rSVD was evaluated separately, and only

fully audited settings support numerical-agreement claims; faster settings are labelled workflow-only. OPLS and kernel PLS met the prespecified tolerances in a separate deterministic reliability study. Same-code execution ablations and direct PLS-SVD/SIMPLS shape comparisons are reported in Supplementary Tables S7a-S7b.

Repeated outer partitions showed that predictive variation exceeded timing variation. Among 46 evaluated family-dataset selections, 24 (52.2%) occurred at a tested-grid boundary and another nine were response-rank limited; thus 33/46 (71.7%) were constrained rather than unconstrained optima. They are reported as best within the evaluated grid (Supplementary Table S14).

3.1 Repeated comparison with an independent SIMPLS implementation

The strict comparison completed all 108 planned runs: nine datasets, two output profiles, two implementations, and three fresh-process repetitions. Accuracy was identical for every pair. With minimum common prediction outputs, fastPLS was faster on four datasets and `pls::simpls.fit` on five; the largest fastPLS advantage was 1.53-fold on GTEx v8. Under ordinary public workflows, fastPLS was faster on five datasets, including 2.41-fold on CIFAR-100, 3.35-fold on Retina, and 4.84-fold on Tabula Muris. Corresponding accuracies were 0.8739 (8,739/10,000; Wilson 95% CI 0.8672-0.8803), 0.9678 (21,684/22,406; 0.9654-0.9700), and 0.8006 (40,077/50,059; 0.7971-0.8041). These timing profiles answer different questions and are not pooled (Figure 2; Supplementary Tables S10a-S10b).

Workflow gains partly reflected output policy. On CIFAR-100, the compact fastPLS fit object was 1.38 MB versus 7,778 MB for ordinary `pls::simpls.fit`. Median complete-process peak RSS was 2.20 versus 13.42 GB; from nearly identical 1.91-GB pre-fit baselines, the corrected peak increments were 0.294 versus 11.51 GB. The theoretical largest retained dense object was the 0.586-MB final coefficient matrix for fastPLS and a 3,814.7-MB fitted or residual response path

for `pls::simpls.fit`. These values describe the specified complete workflow, not isolated algorithmic workspace. When both methods retained complete coefficient paths, fit objects were 59.30 and 58.60 MB, corrected increments were 69.5 and 127.0 MB, and the speed-up was 1.18-fold. The broader package panel remains a workflow comparison with implementation-specific outputs (Supplementary Tables S10c-S10e).

Figure 2. Repeated float64 single-CPU SIMPLS timing against `pls::simpls.fit`. Points are median fit-plus-prediction times; bars are IQRs from $n = 3$ fresh-process repetitions per method-dataset pair. Left: complete coefficient paths and final predictions with dense scores, loadings, and fitted arrays suppressed. Right: ordinary public objects. Accuracy was identical in every argmax pair; full package-by-dataset values are in Supplementary Tables S10a-S10e.

3.2 Internal acceleration and low-rank solver choice

Same-code ablations on CIFAR-100, MetRef, PRISM, and Retina produced zero endpoint-metric differences and identical classifications. The effects were not uniformly faster: cached deflation products changed time by 0.94-1.02-fold, conditional X-transpose-X caching by 0.99-1.00-fold, incremental coefficients by 0.99-1.07-fold, and compact prediction by 0.97-1.24-fold. Compact prediction reduced incremental RSS by up to 77.7%. Implicit cross-covariance products ranged from a 0.065-fold slowdown to a 6.24-fold gain and reduced RSS by up to 70.6%, with the speed gain confined to the high-response PRISM regime. These results support shape-dependent routing, not universal acceleration (Supplementary Tables S2 and S7a).

A direct matched-shape timing study further separated estimator choice from implementation cost. On one CPU, the SIMPLS/PLS-SVD time ratio ranged from 1.00 to 3.84; on CUDA it ranged from 0.92 to 0.98 across five synthetic matrix regimes. Thus, shape-aware SIMPLS approached one-shot PLS-SVD runtime on the tested CUDA shapes, without implying that the two PLS estimators are statistically identical or that SIMPLS is universally faster (Supplementary Table S7b).

The controlled study completed all 486 CPU/CUDA runs. Automatic rSVD met tolerance at all sample-size, predictor-size, requested-prefix, and cross-covariance-size points, but not throughout the retained-component, class-count, rank, or CUDA response-dimension sweeps; 73 of 276 automatic-route runs were therefore excluded from speed claims. Among qualified points, CUDA first exceeded CPU rSVD at $n = 5,000$ and $p = 2,000$ and was 3.04-5.97-fold faster across the 2-64 MB cross-covariance sweep. CPU implicit products reduced incremental RSS by 29.7-47.5%, but were slower below 32 MB and approximately time-neutral at 32-64 MB. A 56-run Metal diagnostic completed without execution failure, but all 20 Metal rSVD routes were numerically discordant and were quarantined (Supplementary Tables S7c-S7d; Figure S1).

Hardware acceleration remained route and shape dependent. CPU, CUDA, and Metal runtime ratios were summarized only for paired predictions meeting the strict display criteria of absolute endpoint-metric difference ≤ 0.005 and prediction agreement ≥ 0.995 (Figure 3; Supplementary Table S11). These thresholds are deliberately stricter than the broader 0.01/0.99 rSVD qualification limits, because Figure 3 labels selected workflows as near-concordant for direct runtime interpretation, whereas the qualification audit screens approximation across complete component paths and several seeds. The former oversampling-10, one-power CPU setting and rejected CUDA settings remain quarantined. Release 0.99.23 defaults to oversampling 20 and two power iterations, which met 585/585 CPU and 40/40 CUDA checks across five seeds; Metal rSVD remains unqualified. Fixed-control CPU IRLBA remains the deterministic numerical reference. float32 approximately halved stored inputs on MetRef and PRISM but did not uniformly improve runtime, incremental memory, or agreement (Supplementary Table S9).

Figure 3. CPU/accelerator runtime ratios for numerically concordant workflows. Cells are colored only when the absolute selected-endpoint metric difference was at most 0.005 and paired-prediction agreement was at least 0.995; ratios above one favor the accelerator and ratios below one favor CPU. These stricter display thresholds identify near-concordant selected workflows; the broader multi-seed rSVD qualification uses 0.01/0.99 across component paths. Gray cells indicate discordance or missing paired predictions. Full paired values are in Supplementary Table S11.

3.3 Multivariate NMR prediction

NMR comprised 1,200 training and 321 held-out spectra, with 13,000 predictors and 28,355 responses. Predictor columns between 4.6 and 4.8 ppm were zeroed in training and test data as standard water-region preprocessing; responses were unmasked. Training-only one-standard-error selection retained five PLS-SVD and 50 SIMPLS components. These are family-specific predictive settings rather than a matched implementation comparison. The complete candidate grids and executable selection rule are reported in Supplementary Section S17.

At the family-selected settings, independent-test Q^2 used the fixed training-response mean. Accordingly, CUDA PLS-SVD and SIMPLS achieved RMSD 0.001043 (Q^2 0.98916) and 0.000759 (Q^2 0.99425), respectively. SIMPLS had lower median per-spectrum and response-wise error. Figure 4 displays held-out sample AMI-00BP-8 (index 155), whose per-spectrum RMSD under 50-component SIMPLS CUDA rSVD was closest to the median across the 321 held-out spectra. This prespecified descriptive rule did not select the best-predicted or most visually concordant spectrum.

A separate matched solver/backend analysis held family, split, float64 precision, and component count fixed. For PLS-SVD at five components, CPU IRLBA, CPU rSVD, and CUDA rSVD required 264.3, 4.74, and 0.41 s; rSVD predictions had relative error at most $3.75e-11$ against IRLBA. For SIMPLS at 50 components, the times were 350.7, 9.80, and 1.51 s. CPU rSVD relative prediction error was $6.29e-11$; CUDA rSVD error was 0.00566 with correlation 0.999981. All rSVD rows used oversampling 20, two power iterations, and seed 123 and met the prespecified approximate-route tolerances.

The deposited 165-component PLS-SVD/IRLBA workflow required 447.6 s and achieved RMSD 0.000710. It is shown as scientific context for the earlier Nature Communications analysis, not as an implementation-only contrast, because component count, workflow, and hardware differ.

Figure 4. NMR predictive and computational analyses. Panels A-C separate family-selected held-out performance from the historical deposited 165-component workflow, which differs in component count, implementation, and hardware and is not a matched reference. Reported Q^2 is independent-test Q^2 relative to the training-response mean. Panels D-E overlay observed and predicted intensities for held-out sample AMI-00BP-8 (index 155), selected as closest to the median per-spectrum RMSD under 50-component SIMPLS CUDA rSVD, over the full response range and 1.7-0.5 ppm expansion. Panel F reports matched float64 solver/backend resources at fixed family-specific component counts. rSVD used oversampling 20, two power iterations, and seed 123.

3.4 ImageNet-scale supervised representation

Exploratory ImageNet experiment 1 used the current package on 1,000,000 training and 281,167 held-out 1,024-dimensional embeddings. The actual route was label-aware float32 SIMPLS with GPU rSVD and native CUDA LDA, but sequential SIMPLS deflation and score projection remained host-resident; it is therefore hybrid, not fully GPU-resident. rSVD used oversampling 20, two power iterations, and seed 123.

A single shared component-path fit evaluated requested prefixes from 100 to 1,000 components. The 1,000-component row was the prespecified boundary stress point, not a selected optimum. Top-1/top-5 accuracy changed from 0.7806/0.9278 at 100 components to 0.8094/0.9393 at 1,000 components; the latter corresponded to 227,571/281,167 correct top-1 predictions (Wilson 95% CI 0.8079-0.8108). The shared fit required 950.2 s. Peak process RSS was 17236 MB and peak

device memory was 5412 MB (4122 MB above baseline). Because the split was noncanonical and the measurements were single runs, these are exploratory feasibility estimates, not confirmatory accuracy claims.

Figure 5. Exploratory ImageNet matrix-processing feasibility with float32 SIMPLS-LDA. Seed 123 assigned 1,000,000 rows from the pooled 1,281,167-row DINOv2 embedding archive to training and the complementary 281,167 rows to holdout; this was not the canonical ImageNet split. Accuracy and blocked prediction time are shown for requested 100-1,000-component prefixes from one shared fit; 1,000 is a boundary stress point, not an optimum. The route is hybrid: SIMPLS deflation and score projection are host-resident, whereas rSVD range products and LDA use CUDA. Controls were oversampling 20, two power iterations, and seed 123. Values are single-run exploratory feasibility measurements.

4. Discussion

The computational results support a shape-dependent choice rather than one universally preferred PLS family. On one CPU, PLS-SVD was as fast as or faster than SIMPLS across the five matched synthetic shapes (SIMPLS/PLS-SVD time ratio 1.00-3.84), as expected for a one-shot decomposition. On the qualified CUDA shapes, SIMPLS approached or marginally exceeded PLS-SVD speed (ratio 0.92-0.98), while retaining sequential orthogonalization and support for component counts not restricted by response rank. Family selection should nevertheless be based on training-only predictive validation, because PLS-SVD and SIMPLS are different estimators. Compact prediction matters most when the test set, response dimension, or

number of requested prefixes is large: it reduced incremental RSS by up to 77.7% by avoiding dense coefficient and fitted-response paths, but offered little benefit when those outputs were intrinsically small.

The storage route and hardware should likewise follow matrix shape. Implicit cross-covariance products are useful when explicitly storing X -transpose- Y or its deflated intermediates is the memory bottleneck. In the controlled CPU study they reduced incremental RSS by 29.7-47.5%, but were slower when the explicit cross-covariance occupied less than 32 MB and approximately time-neutral at 32-64 MB; the large speed gain occurred only in the high-response PRISM regime. CUDA first surpassed CPU rSVD at $n = 5,000$ in the sample-size sweep and $p = 2,000$ in the predictor-size sweep. These are empirical crossover points on the tested hardware, not portable thresholds: transfer, context creation, synchronization, matrix aspect ratio, and available device memory remain decisive. Metal was disadvantaged by host-assisted stages, conversion and dispatch overhead, and the small integrated-GPU test system; moreover, all 20 Metal rSVD diagnostic routes were numerically discordant and were therefore excluded from speed claims rather than interpreted as valid accelerator comparisons.

Solver choice separates exploratory acceleration from confirmatory analysis. The qualified rSVD defaults (oversampling 20 and two power iterations) should be checked across several prespecified seeds when a result is scientifically consequential; users should compare predictions, endpoint metrics, and, where relevant, latent subspaces, without selecting the seed that performs best on held-out test labels. Material seed variation, diagnostic failure, ill-conditioning, very small samples, or a requirement for deterministic reproducibility are indications to use CPU IRLBA. When a selected component count reaches the upper grid boundary, the result is the best within that grid rather than an optimum; the training-only grid

should be extended if rank and computation permit, or a one-standard-error rule and the boundary limitation should be reported. Nonlinear kernel PLS requires an n -by- n Gram matrix: the matrix alone occupies approximately 0.8, 5.0, and 20.0 GB in float64 at $n = 10,000$, 25,000, and 50,000, before copies and workspaces. It should therefore be restricted to moderate n or replaced by a linear route or an externally approximated kernel. Unsupported backend/solver combinations stop explicitly; experimental or unqualified approximate routes warn and record diagnostics, and fastPLS does not silently replace the requested estimator. Table 1 summarizes these practical decisions.

Table 1. Practical interpretation of the computational evidence and recommended fastPLS route.

Question	Observed trigger or constraint	Recommended action
PLS family	CPU matched shapes favored one-shot PLS-SVD; qualified CUDA shapes placed sequential SIMPLS near parity. PLS-SVD is response-rank limited.	Use training-only prediction to choose the estimator. Prefer PLS-SVD for a low-rank, one-shot path; use SIMPLS when sequential components or components beyond response rank are required.
Compact prediction	Large test n , large q , or many requested prefixes; up to 77.7% lower incremental RSS in the ablation.	Retain latent factors and block predictions. Dense coefficient, score, loading, or fitted paths should be requested only when scientifically needed.
Implicit products	Memory pressure from X -transpose- Y ; slower below 32 MB and near time-neutral at 32-64 MB in the controlled CPU sweep.	Use primarily to reduce memory for large p by q products, especially high q . Keep the explicit route for small cross-covariance matrices unless profiling supports otherwise.
CUDA	First qualified crossover at $n = 5,000$ or $p = 2,000$ in separate one-factor sweeps on the tested NVIDIA system.	Benchmark the intended shape including transfer and synchronization. Interpret speed only when paired predictions meet numerical tolerances.
Metal	Host-assisted stages and all 20 diagnostic rSVD routes were numerically discordant on the tested Apple system.	Treat Metal rSVD as experimental; use deterministic CPU execution for confirmatory work until the route is qualified.
IRLBA or rSVD	Need for deterministic reproducibility, diagnostic failure, material seed variation, ill-conditioning, or small problems where approximation saves little time.	Use CPU IRLBA for confirmation. Use qualified rSVD controls for accelerated exploratory or production workflows and retain solver diagnostics.
rSVD seeds	Scientifically consequential approximate fit or a new matrix regime.	Run several prespecified seeds and compare predictions, endpoint metrics, and subspaces. Do not choose a seed using held-out test performance; switch to IRLBA if conclusions vary.
Boundary component	The selected component is the largest value evaluated or is constrained by rank.	Report 'best within the evaluated grid'; extend the training-only grid where feasible or use a prespecified parsimonious rule such as one standard error.
Nonlinear kernel PLS	The float64 Gram matrix alone is $8n$ -squared bytes: about 0.8 GB at 10,000, 5.0 GB at 25,000, and 20.0 GB at 50,000 samples.	Restrict to moderate n after accounting for copies and workspaces. For larger n , use linear PLS or an externally validated kernel approximation.

Question	Observed trigger or constraint	Recommended action
Unsupported route	Unavailable backend/solver/family combination or unqualified approximate controls.	Unavailable combinations produce an error. Experimental or unqualified routes warn and record diagnostics; no silent estimator substitution is permitted.

5. Conclusions

fastPLS reduces avoidable computation and storage along the sequential SIMPLS component path while preserving the deterministic de Jong estimator within the prespecified numerical tolerances. OPLS, kernel PLS, approximate low-rank solvers, and accelerator backends extend this core contribution under route-specific validation; deterministic float64 CPU SIMPLS remains the confirmatory reference.

Ethics statement

This software study used public, simulated, or previously collected de-identified data; source-study ethics are reported in the cited publications.

CRedit authorship contribution statement

Dupe Ojo: Investigation, validation, data curation, visualization, writing - review and editing.

Alessia Vignoli: Investigation, validation, data curation, visualization, writing - review and

editing. Stefano Cacciatore: Conceptualization, methodology, software, supervision, project

administration, writing - original draft, writing - review and editing. Leonardo Tenori:

Conceptualization, methodology, supervision, writing - review and editing. This draft allocation must be confirmed by all authors before submission.

Acknowledgements

The authors thank the University of Cape Town's ICTS High Performance Computing team for providing a high-performance computing facility for this study (<https://ucthpc.uct.ac.za/>).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

Code and benchmark outputs are available at <https://github.com/tkcaccia/fastPLS>; reusable components are at <https://github.com/tkcaccia/kodama-cpp>. Historical quantitative results remain tied to fastPLS 0.99.6, base commit 6e50bd318f20289101f6b723953830aefa8b95d6, and source-archive SHA-256 c868770fee196af24bfb15b4da9fad1d7765deaf68c0ef1fd3e5a15670ecaa85; they are not relabelled as reruns. The reviewed interface is fastPLS 0.99.25. Its exact source archive fastPLS_0.99.25.tar.gz has SHA-256 74e134ef22d591c3ac5b06910880ea56dac1970b5277cc6c1c25d3dcff3d9e58. The definitive CPU/CUDA multi-seed rSVD qualification was generated with 0.99.23 before the documentation and Metal direction-refresh correction; the active CPU/CUDA numerical path and qualified controls are unchanged in 0.99.25. Analysis-specific scripts and archive digests are reported in Supplementary Table S15. The accepted source, vignette, reference manual, benchmark scripts, and definitive key tables will be deposited as one immutable tagged release with a persistent archive identifier; until that deposit is minted, the SHA-256-identified source archive is the immutable review object.

References

- [1] Wold S, Sjostrom M, Eriksson L. PLS-regression: a basic tool of chemometrics. *Chemom Intell Lab Syst.* 2001;58:109-130. [https://doi.org/10.1016/S0169-7439\(01\)00155-1](https://doi.org/10.1016/S0169-7439(01)00155-1).
- [2] Geladi P, Kowalski BR. Partial least-squares regression: a tutorial. *Anal Chim Acta.* 1986;185:1-17. [https://doi.org/10.1016/0003-2670\(86\)80028-9](https://doi.org/10.1016/0003-2670(86)80028-9).
- [3] Barker M, Rayens W. Partial least squares for discrimination. *J Chemom.* 2003;17:166-173. <https://doi.org/10.1002/cem.785>.

- [4] Bertini I, Cacciatore S, Jensen BV, et al. Metabolomic NMR fingerprinting to identify and predict survival of patients with metastatic colorectal cancer. *Cancer Res.* 2012;72:356-364. <https://doi.org/10.1158/0008-5472.CAN-11-1543>.
- [5] Cacciatore S, Wium M, Licari C, et al. Inflammatory metabolic profile of South African patients with prostate cancer. *Cancer Metab.* 2021;9:29. <https://doi.org/10.1186/s40170-021-00265-6>.
- [6] Elebo N, et al. Serum metabolomic and lipoprotein profiling of pancreatic ductal adenocarcinoma patients of African ancestry. *Metabolites.* 2021;11:663. <https://doi.org/10.3390/metabo11100663>.
- [7] Vignoli A, Cacciatore S, Tenori L. Deriving three one-dimensional NMR spectra from a single experiment through machine learning. *Nat Commun.* 2025;16:10159. <https://doi.org/10.1038/s41467-025-65294-x>.
- [8] Cacciatore S, Tenori L, Luchinat C, Bennett PR, MacIntyre DA. KODAMA: an R package for knowledge discovery and data mining. *Bioinformatics.* 2017;33:621-623. <https://doi.org/10.1093/bioinformatics/btw705>.
- [9] Cacciatore S, Luchinat C, Tenori L. Knowledge discovery by accuracy maximization. *Proc Natl Acad Sci USA.* 2014;111:5117-5122. <https://doi.org/10.1073/pnas.1220873111>.
- [10] Wegelin JA. A survey of partial least squares methods, with emphasis on the two-block case. University of Washington; 2000.
- [11] de Jong S. SIMPLS: an alternative approach to partial least squares regression. *Chemom Intell Lab Syst.* 1993;18:251-263. [https://doi.org/10.1016/0169-7439\(93\)85002-X](https://doi.org/10.1016/0169-7439(93)85002-X).
- [12] Trygg J, Wold S. Orthogonal projections to latent structures (O-PLS). *J Chemom.* 2002;16:119-128. <https://doi.org/10.1002/cem.695>.
- [13] Lindgren F, Geladi P, Wold S. The kernel algorithm for PLS. *J Chemom.* 1993;7:45-59. <https://doi.org/10.1002/cem.1180070104>.
- [14] Rosipal R, Trejo LJ. Kernel partial least squares regression in reproducing kernel Hilbert space. *J Mach Learn Res.* 2001;2:97-123.
- [15] Baglama J, Reichel L. Augmented implicitly restarted Lanczos bidiagonalization methods. *SIAM J Sci Comput.* 2005;27:19-42. <https://doi.org/10.1137/04060593X>.
- [16] Halko N, Martinsson PG, Tropp JA. Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Rev.* 2011;53:217-288. <https://doi.org/10.1137/090771806>.
- [17] Fisher RA. The use of multiple measurements in taxonomic problems. *Ann Eugen.* 1936;7:179-188.
- [18] McLachlan GJ. Discriminant Analysis and Statistical Pattern Recognition. Wiley; 1992.
- [19] Mevik BH, Wehrens R. The pls package: principal component and partial least squares regression in R. *J Stat Softw.* 2007;18:1-23. <https://doi.org/10.18637/jss.v018.i02>.

- [20] Assfalg M, et al. Evidence of different metabolic phenotypes in humans. *Proc Natl Acad Sci USA*. 2008;105:1420-1424. <https://doi.org/10.1073/pnas.0705685105>.
- [21] Stoeckius M, et al. Simultaneous epitope and transcriptome measurement in single cells. *Nat Methods*. 2017;14:865-868. <https://doi.org/10.1038/nmeth.4380>.
- [22] GTEx Consortium. The GTEx Consortium atlas of genetic regulatory effects across human tissues. *Science*. 2020;369:1318-1330. <https://doi.org/10.1126/science.aaz1776>.
- [23] Ghandi M, et al. Next-generation characterization of the Cancer Cell Line Encyclopedia. *Nature*. 2019;569:503-508. <https://doi.org/10.1038/s41586-019-1186-3>.
- [24] Hoadley KA, et al. Cell-of-origin patterns dominate the molecular classification of 10,000 tumors from 33 types of cancer. *Cell*. 2018;173:291-304.e6. <https://doi.org/10.1016/j.cell.2018.03.022>.
- [25] Macosko EZ, Basu A, Satija R, et al. Highly parallel genome-wide expression profiling of individual cells using nanoliter droplets. *Cell*. 2015;161:1202-1214. <https://doi.org/10.1016/j.cell.2015.05.002>.
- [26] Tabula Muris Consortium. Single-cell transcriptomics of 20 mouse organs creates a Tabula Muris. *Nature*. 2018;562:367-372. <https://doi.org/10.1038/s41586-018-0590-4>.
- [27] Corsello SM, et al. Discovering the anticancer potential of non-oncology drugs by systematic viability profiling. *Nat Cancer*. 2020;1:235-248. <https://doi.org/10.1038/s43018-019-0018-6>.
- [28] Deng J, et al. ImageNet: a large-scale hierarchical image database. *Proc IEEE CVPR*. 2009:248-255. <https://doi.org/10.1109/CVPR.2009.5206848>.
- [29] Oquab M, Darcet T, Moutakanni T, Vo HV, Szafraniec M, Khalidov V, et al. DINOv2: learning robust visual features without supervision. *Trans Mach Learn Res*. 2024. <https://openreview.net/forum?id=a68SUt6zFt>; arXiv:2304.07193.
- [30] Krizhevsky A, Hinton G. Learning multiple layers of features from tiny images. Technical report. University of Toronto; 2009. <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- [31] Chen RJ, Ding T, Lu MY, et al. Towards a general-purpose foundation model for computational pathology. *Nat Med*. 2024;30:850-862. <https://doi.org/10.1038/s41591-024-02857-3>.
- [32] Xu H, Usuyama N, Bagga J, et al. A whole-slide foundation model for digital pathology from real-world data. *Nature*. 2024;630:181-188. <https://doi.org/10.1038/s41586-024-07441-w>.